

Program Transformations

Outline

- Program Transformations
 - Bytecode
 - AST

Program Transformations

Why might we want to transform a program?

- Optimizations
- Obfuscations
- Debugging
- Performance Monitoring
- ...
- We'll learn more in this course!

AST Transformations

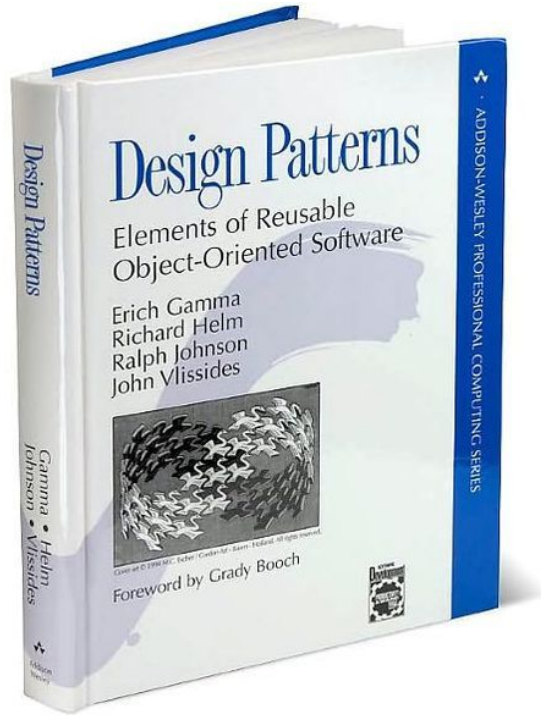
AST Transformations

Example: inject performance logging for each method (print time taken for each method)

- Find all method declarations
- Create new statements
 - Store start time
 - Store end time
 - Print end - start
- Insert them into the method declaration
 - In the proper location!

Traversing the AST tends to be laborious and error prone. This is largely due to the process relying on recursion and frequent type checking to attain your goal

Design Patterns



- Reusable solutions to common software design problems
- Provide a way to structure your code in a way that is flexible, maintainable, and efficient
- *“All well-structured object-oriented architectures are full of patterns. Indeed, one of the ways that I measure the quality of an object-oriented system is to judge whether or not its developers have paid careful attention to the common collaborations among its objects.”*

Visitor Design Pattern

- **Visitor Design Pattern** commonly used for tree transformations
- Allows you to separate the operations / transformations from the traversal process
- Trees often have different types of nodes. The Visitor pattern makes it easy to define different operations for each node type by using method overloading in the visitor.
- Each operation is encapsulated in a "visitor," which visits different node types (like addition or multiplication) and processes them.

JavaParser Library

- Allows you to interact with Java source code as an AST
- Provides mechanism to navigate the tree with *Visitor Support*
 - Allows you to write operations without needing to write traversal code

A Simple Visitor

Let's create a visitor to print the name of each method in a program

You can also pass a value to the visitor!

- Useful to save or load state

A Simple Modifying Visitor

Let's write a visitor to change variable names (declarations and references) to all caps in Method `decls`

```
public class Vars {  
  
    public void decls() {  
        int foo = 15;  
        int helloworld = 20;  
        int added = foo + helloworld;  
    }  
  
    public void dontchange() {  
        int lower = 15;  
        int stillLower = 10;  
        int lowercase = lower + stillLower;  
    }  
}
```

JavaParser library

Basic Workflow:

1. Parse the source into an AST
 - a. `JavaParser.parse(sourceCode);`
2. Transform the AST
 - a. Using a Visitor, traverse the AST and perform a transformation
3. Write the modified AST
 - a. Call `toString()` on the AST root (`CompilationUnit`) to get the source

Bytecode Transformations

Review: Java Bytecode

```
public class Ex0 {  
  
    public int add(int x) {  
        return x + 1;  
    }  
  
    public void start() {  
        int z = add(1);  
    }  
}
```

public int add(int);
Code:
0: iload_1
1: iconst_1
2: iadd
3: ireturn

LocalVariableTable:
Start Length Slot Name
0 4 0 this LEx0;
0 4 1 x I

public void start();
Code:
0: aload_0
1: iconst_1
2: invokevirtual #2 // Method add:(I)I
5: istore_1
6: return

LocalVariableTable:
Start Length Slot Name
0 7 0 this LEx0;
6 1 1 z I

Operand Stack



Local Variable Table



Example 2

```
public class Ex1 {  
  
    public void foo(int x) {  
        int y = x + 1;  
    }  
  
    public void baz() {  
        int z = 99;  
        foo(z);  
    }  
}
```

```
public void foo(int);
```

Code:

```
0: iload_1  
1: iconst_1  
2: iadd  
3: istore_2  
4: return
```

LocalVariableTable:

Start	Length	Slot	Name
0	5	0	this LEx1;
0	5	1	x I
4	1	2	y I

```
public void baz();
```

Code:

```
0: bipush      99  
2: istore_1  
3: aload_0  
4: iload_1  
5: invokevirtual #2 // Method foo:(I)V  
8: return
```

LocalVariableTable:

Start	Length	Slot	Name
0	9	0	this LEx1;
3	6	1	z I

Operand Stack



Local Variable Table



ASM

Java Library for Bytecode manipulation

Basic Workflow:

1. Read the bytecode
 - a. `ClassReader`
2. Transform the bytecode
3. Write the modified bytecode
 - a. `ClassWriter`



Bytecode Transformation

Increment every constant by 1 in the main method

```
public class Constants {  
  
    public static void dontModify(int q) {  
        int x = 100;  
        int y = 10;  
        int z = x + y + 1;  
    }  
  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
        int c = 30;  
  
        dontModify(100);  
    }  
}
```

```
public static void dontModify(int);
```

Code:

```
0: bipush      100  
2: istore_1  
3: bipush      10  
5: istore_2  
6: iload_1  
7: iload_2  
8: iadd  
9: iconst_1  
10: iadd  
11: istore_3  
12: return
```

```
public static void main(java.lang.String[]);
```

Code:

```
0: bipush      10  
2: istore_1  
3: bipush      20  
5: istore_2  
6: bipush      30  
8: istore_3  
9: bipush      100  
11: invokestatic #7    // Method dontModify:(I)V  
14: return
```

Bytecode vs AST transformations

Transformation: Wrap all calls to a method `foo` in a try-catch block as precisely as possible.

An aside: Java Bytecode for Exception Handling

When an exception occurs, the JVM stores the current PC and scans the **Exception Table** for a range that contains the PC and goes to the target

Bytecode vs AST transformations

Transformation: Wrap all calls to a method `foo` in a try-catch block as precisely as possible.

Consider performing this transformation at both the **bytecode** and **AST** level for both examples:

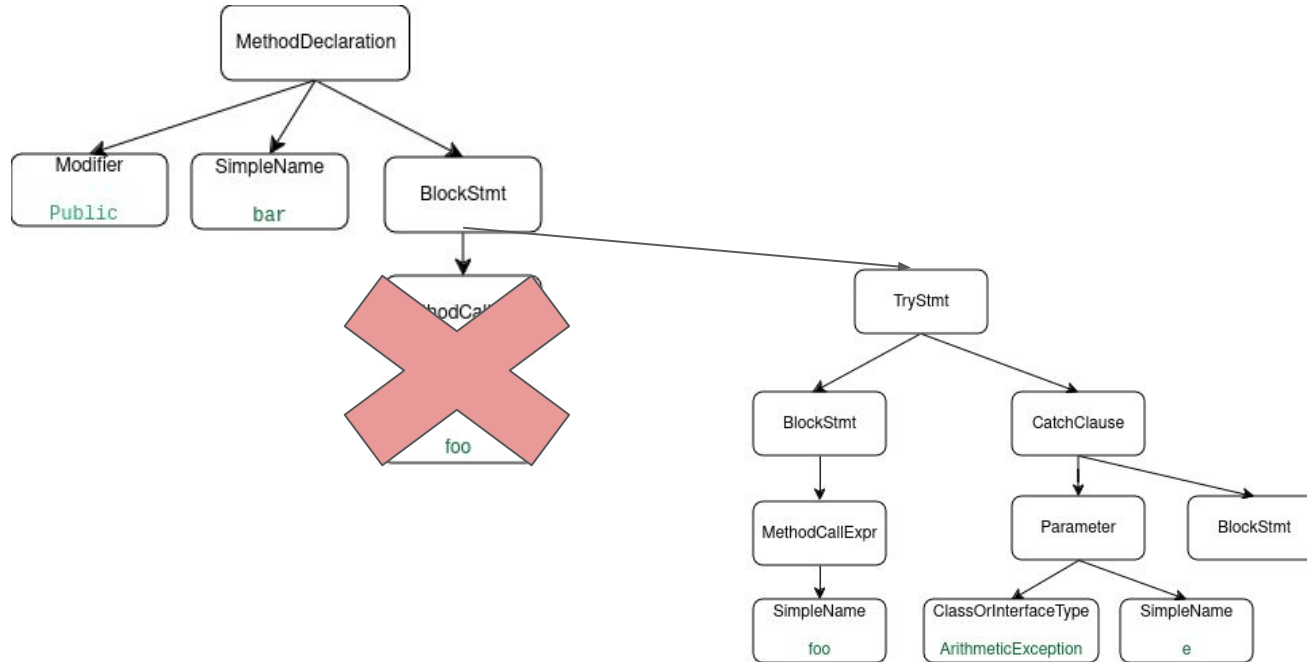
Example 1: **public void** **bar**(**int** x) { foo(); }

Example 2: **public void** **bar**(**int** x) { int x = foo(a) + bar(a); }

Work through it by hand considering err cases. Which representation do you prefer?

AST try-catch Transformation

```
public void bar(int x) {  
    foo();  
}
```



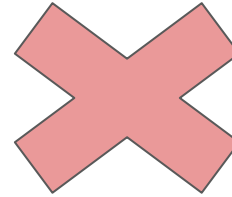
AST try-catch Transformation

Failure cases:

```
public void bar(int x) {  
    int z = foo();  
    z = z + 1;  
}
```



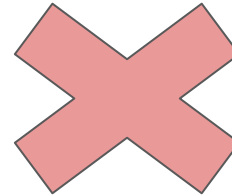
```
public void bar(int x) {  
    try {  
        int z = foo();  
    } catch (Exception e) {  
  
    }  
    z = z + 1;  
}
```



```
public void bar(int x) {  
    if (foo()) {  
        ...  
    }  
}
```



```
public void bar(int x) {  
    try {  
        if (foo()) {  
            ...  
        }  
    }  
}
```



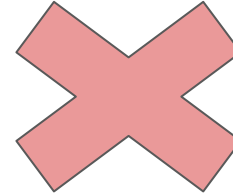
AST try-catch Transformation

Failure cases:

```
public void bar(int x) {  
    for (int i=0; i<foo(); i++)  
        ..  
}
```



```
public void bar(int x) {  
    try {  
        for (int i=0; i<foo(); i++) ..  
    } catch (Exception e) {  
    }  
}
```



```
public void bar(int x) {  
    baz(foo());  
}
```

...

Bytecode try-catch Transformation

1. Add statements after invokevirtual:
 - a. goto x
 - b. astore_x //store exception in LVT
2. Add entry to exception table
3. modify labels on following statements

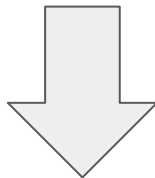
```
public void bar();
```

```
Code:
```

```
0: aload_0
```

```
1: invokevirtual #2           // Method foo():V
```

```
4: return
```



```
public void bar();
```

```
Code:
```

```
0: aload_0
```

```
1: invokevirtual #2           // Method foo():V
```

```
4: goto          8
```

```
7: astore_2
```

```
8: return
```

Bytecode try-catch Transformation

Failure cases of transformation on an AST:

- Output of `foo` saved in a variable
- `foo` is the condition in an if-statement
- `foo` is the stopping condition in a for-loop

Would these be problematic in bytecode transformations?

Summary

- We learned how to transform programs over two different representations
- HW2 Released
 - AST and Bytecode transformations
 - Due Monday September 28th
 - It is a difficult and time consuming assignment. Start early!